migs flow
WEB DESIGN

FRONT-END STANDARDS

Web Development Handbook

File structure, clean URLs, code rules, images, fonts, performance, accessibility, SEO, forms and launch, for every MigsFlow website.

VERSION	UPDATED	TARGETS
2.0	November 6, 2026	Lighthouse 98+ · WCAG 2.2 AA

Contents

0. Hard Rules at a Glance
1. Project Structure and Clean URLs
2. Separation of Concerns
3. Approved Exceptions
4. Global vs. Page-Specific Code
5. Mobile-First CSS and Responsive Design
6. Images
7. Fonts
8. JavaScript and Third-Party Code
9. CSS Performance
10. Performance Targets
11. Accessibility (WCAG 2.2 AA)
12. SEO and Launch Basics
13. Forms and Third-Party Integrations
14. Domains, Hosting, Security and Git
15. Content Integrity
16. MigsFlow Build Workflow
17. Launch Checklist
18. Instructions for AI Coding Agents

What's new in 2.0

- Clean URLs: page folders replace the `html/` folder (no more `/html/` in addresses).
- Performance target raised to 98+ mobile on every page, with Core Web Vitals budgets.
- Full WCAG 2.2 AA accessibility section, including the new 2.2 criteria.
- New sections: SEO and launch, forms and integrations, domains and Git, content integrity.
- Approved-exceptions list, design tokens, a launch checklist, and lessons from 2026 projects.
- This handbook ships in every project as `CLAUDE.md`, so AI agents read it automatically.

0. Hard Rules at a Glance

1. `index.html` stays at the root. Every other page is a folder with its own `index.html`, giving clean URLs like `/about/`. No `html/` folder.
2. No inline CSS or JavaScript in HTML. The only exceptions are listed in Section 3.
3. Mobile-first CSS: base styles are for phones, and `min-width` media queries enhance larger screens.
4. Shared code goes in `global.css` and `global.js`. Page files hold only page-specific code.
5. Fonts are self-hosted in `fonts/`. Never use Google Fonts or another font CDN.
6. Every raster image has a WebP version, a JPG fallback when needed, and a `-m` mobile crop when useful. All images get `decoding="async"` and explicit `width` and `height`. Images live in subfolders of `images/` (e.g. `images/hero/`), never loose at its root.
7. Lazy-load images below the fold only. Never lazy-load the hero or logo.
8. SVG icons only. No Font Awesome and no jQuery.
9. **Lighthouse mobile: Performance 98+ and Accessibility, Best Practices and SEO all 100 on every page.**
10. Accessibility meets **WCAG 2.2 Level AA**.
11. Every page has a unique title (**50 characters or fewer, never over 60**), meta description, canonical URL, one `h1` and Open Graph tags.
12. Never put API keys, passwords, or secrets in front-end code or in the repo.
13. Reviews, testimonials, stats, and credentials must be real. Never invent them.
14. Every site has a **back-to-top button** fixed to the bottom-right corner (Section 4).

1. Project Structure and Clean URLs

Required structure

```
project-root/
├── index.html                ← homepage → domain.com/
├── about/
│   └── index.html            ← domain.com/about/
├── services/
│   ├── index.html            ← domain.com/services/
│   └── move-out-cleaning/
│       └── index.html        ← domain.com/services/move-out-cleaning/
├── contact/
│   └── index.html            ← domain.com/contact/
├── privacy-policy/
│   └── index.html
├── 404.html                  ← custom "page not found" page
├── css/
│   ├── global.css
│   ├── styles.css            ← homepage only
│   ├── about.css
│   └── critical.css          ← only when needed
├── javascript/
│   ├── global.js
│   ├── script.js             ← homepage only
│   └── about.js
├── images/                   ← subfolders only (Section 6), nothing loose
│   ├── brand/                ← logos, wordmarks, OG image
│   ├── hero/
│   ├── about/
│   ├── services/
│   ├── icons/                ← SVG icons
│   └── decor/                ← textures, stamps, illustrations
├── fonts/
├── favicon.ico, favicon.svg, apple-touch-icon.png, icon-192.png, icon-512.png
├── site.webmanifest
├── robots.txt
├── sitemap.xml
├── _redirects                ← Netlify only (Section 12)
├── .gitignore
├── documentation/            ← handbook PDF, client-safe docs
└── CLAUDE.md                 ← this handbook
```

Why folders instead of `html/about.html`

A page saved as `about/index.html` loads at `domain.com/about/`, with no `/html/` and no `.html` in the address. This works on **Netlify, GitHub Pages, and every other static host with no settings or server code**, and it works locally with any local server (Live Server, `python3 -m http.server`).

Path rules

Use **relative paths**. They work on a custom domain *and* on GitHub Pages preview links like `username.github.io/Project-Name/`, where links starting with `/` break.

File	CSS link	Link to About	Link to Home
`index.html`	`css/global.css`	`about/`	`./`
`about/index.html`	`../css/global.css`	`./`	`../`
`services/move-out-cleaning/index.html`	`../../css/global.css`	`../../about/`	`../../`

- Folder links always end with a slash: `about/`, not `about` or `about/index.html`.
- Folder and file names are lowercase, use hyphens, and describe the page (`move-out-cleaning`, not `page2` or `Move_Out`).
- The **canonical tag and sitemap** use full absolute URLs with the trailing slash: `https://domain.com/about/`.
- When converting an older site that used `html/`, add redirects from the old URLs (Section 12).

2. Separation of Concerns

Hard rule: never write CSS or JavaScript inside an HTML file.

- HTML holds semantic structure and content.
- CSS holds presentation, layout, responsive behavior and visual states.
- JavaScript holds interactivity and behavior.
- No `style=""` attributes, no `<style>` blocks, no inline `<script>` code, and no `onclick=""` handlers.

3. Approved Exceptions

These are the **only** allowed inline snippets. Each one should have a short HTML comment explaining it.

1. **JSON-LD structured data** (`<script type="application/ld+json">`). It's data, not code, and Google expects it in the page.
2. **Third-party embed snippets** that cannot work any other way (booking widgets, Jobber/HoneyBook forms, analytics loaders). Prefer loading them from `global.js` when the vendor allows it.
3. **Deferred stylesheet pattern** (`media="print" onload="this.media='all'"` plus a `<noscript>` fallback), and only when critical CSS is proven to raise the score (Section 9).

4. **A tiny theme-init script in** `<head>` for sites with a dark/light toggle, to prevent a flash of the wrong theme. Keep it in its own file (`javascript/theme-init.js`) when possible.

If a requested feature conflicts with a hard rule, explain the conflict before building the exception.

4. Global vs. Page-Specific Code

global.css

- `@font-face` declarations, at the top.
- **Design tokens** on `:root`: colors, font families, a type scale, spacing, radius, shadows, and transition timing.
- Reset and base typography.
- Navigation, mobile navigation and footer.
- Buttons, forms, cards, containers, section spacing, and any component used on more than one page.
- Accessibility utilities: `.visually-hidden`, `.skip-link`, focus styles, and `prefers-reduced-motion` rules.
- The back-to-top button (required on every site, see below).

Back-to-top button (required on every site)

- A round button fixed to the **bottom-right corner** in the site's accent color, with an upward arrow. It goes on every page, just before `</body>`.
- It's a link to the top of the page (`href="#top"` pointing at the hero or `<body>` id), so it still works without JavaScript.
- It's hidden at the top of the page and fades in once the visitor scrolls past about 60% of the first screen. The logic lives in `global.js`.
- It has hidden link text (`<span class="visually-hidden">Back to top</span>`), a visible focus style, a tap target of at least 44px, and the same bottom-to-top hover fill as the site's other buttons.
- Keep other floating controls (slideshow pause buttons, chat widgets) out of the bottom-right corner so nothing overlaps it.

```
<a class="back-to-top" href="#top">
    <span class="visually-hidden">Back to top</span>
    <svg aria-hidden="true" width="20" height="20" viewBox="0 0 20 20"><path d="M10 16V4m-5 5 5-5 5
5" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-
linejoin="round"/></svg>
</a>
```

Page-specific CSS

- `styles.css` is the homepage. `about.css`, `contact.css` and the like hold only what is unique to that page.
- Before adding a rule to a page file, check whether the pattern already exists or will repeat. If it will, put it in `global.css`.

JavaScript

- `global.js` holds the nav toggle, shared form handling, reveal animations, the back-to-top button and footer year.
- `script.js` and page files hold only that page's behavior.
- Never duplicate a function across page files.
- Load every script with `defer` at the end of `<head>` or the bottom of `<body>`.

Design tokens example

```
:root {
  --color-ink: #17131a;
  --color-accent: #0b3cff;
  --color-accent-text: #0077c2;   /* passes 4.5:1 behind white text */
  --font-display: 'Bodoni Moda', Georgia, serif;
  --font-body: 'Poppins', Arial, sans-serif;
  --space-section: clamp(4rem, 8vw, 7.5rem);
  --radius: 0.75rem;
}
[data-theme="light"] { /* override tokens only */ }
```

Naming

- Use BEM-style classes: `.hero`, `.hero__title`, `.hero__btn--primary`.
- Use descriptive names. Never use `box1`, `thing` or `temp`.

5. Mobile-First CSS and Responsive Design

Hard rule: base CSS is the phone layout. `min-width` media queries add enhancements for larger screens. Never build desktop first and shrink it down with `max-width`.

- Mobile-first does not mean mobile-only. Every page must work and look balanced on phones, tablets, laptops, desktops and ultrawide screens.
- Prefer fluid techniques: `clamp()`, `min()`, `max()`, percentages, `max-width`, Flexbox and Grid.
- Add a breakpoint only when the layout actually needs to change, not because a device exists at that width.
- Test the in-between widths (e.g. 360, 390, 600, 768, 900, 1024, 1280, 1440, 1920) for overflow, awkward wrapping and layout jumps.
- No horizontal scrolling at any width down to 320px. This is also WCAG 1.4.10, Reflow.
- Keep a 16px minimum side gutter on mobile.
- Body text is at least 16px.

6. Images

Hard rule: every raster image has a WebP version as the primary format, a JPG fallback when appropriate, and a `-m` mobile crop when it reduces file size or improves framing.

Folder organization

Hard rule: no image files sit loose at the root of `images/`. Every image goes in a subfolder named for where it's used. Use the section of the site (`hero/`, `about/`, `services/`, `gallery/`, `testimonials/`) or the type of asset (`brand/`, `icons/`, `decor/`). A page that has its own images gets its own subfolder (e.g. `images/contact/`). Keep each image's desktop, `-m`, WebP and JPG versions together in the same subfolder. Keep original, unoptimized source files out of `images/`: put them in `documentation/source-images/` or outside the repo.

Sizing and compression

- Crop to about **2× the displayed size**. For example, an image shown at 551×250 is saved at 1102×500, and its mobile version shown at 292×213 is saved at 584×426.
- Mobile images should be about **20–40 KB**. Background images should stay **under 100 KB**.
- Compress before adding (Squoosh, compressor.io or sharp) and convert to WebP.
- For the hero on mobile, crop the center of the desktop image rather than shrinking the whole thing.
- File names are descriptive and hyphenated, for example `kitchen-deep-clean.webp` and `kitchen-deep-clean-m.webp`. This helps SEO too.

Markup

```
<picture>
  <source media="(max-width: 600px)" srcset="images/services/kitchen-m.webp" type="image/webp">
  <source srcset="images/services/kitchen.webp" type="image/webp">
  <img src="images/services/kitchen.jpg" alt="Spotless white kitchen after a deep clean"
       width="1102" height="500" loading="lazy" decoding="async">
</picture>
```

- Every `<img>` has `width`, `height` and `decoding="async"`.
- Below the fold, add `loading="lazy"`.
- **Never** lazy-load above-the-fold images. Give the hero image `fetchpriority="high"` and preload it: `<link rel="preload" as="image" href="images/hero/hero-m.webp" media="(max-width: 600px)">`
- **Background images:** use a `<picture>` positioned absolutely behind the section content (`object-fit: cover`) instead of CSS `background-image`, so it can be lazy-loaded, decoded asynchronously and swapped for mobile.
- **Carousels and sliders:** load only the first visible slide's images up front. Lazy-load the rest. JavaScript that swaps `data-src` must still choose a mobile-sized file on mobile.

Alt text

- Meaningful images get alt text describing what the image shows and why it matters, under about 125 characters, and without starting with "image of".
- Decorative images get `alt=""` **and** `aria-hidden="true"`.
- Logos use the business name as their alt text.

7. Fonts

Hard rule: fonts are self-hosted in `fonts/`. Never use the Google Fonts CDN.

- Download fonts with **google-webfonts-helper** (gwfh.mranftl.com). Choose the **Latin subset only**, and use WOFF2.
- Use only the weights you need. Usually that's regular, 700 and italic.
- Declare `@font-face` at the top of `global.css` with `font-display: swap`.
- Preload only the one or two font files used above the fold: `<link rel="preload" href="fonts/poppins-v24-latin-regular.woff2" as="font" type="font/woff2" crossorigin>`
- Pick a similar system fallback font (e.g. `Arial` for Poppins, `Georgia` for serif display faces) to reduce layout shift.
- If a heading jumps to two lines when the web font loads, adjust its size or letter-spacing so the line wraps the same way with both fonts. This prevents CLS.

8. JavaScript and Third-Party Code

- Write vanilla JavaScript. No jQuery or frameworks unless the project truly requires them.
- `defer` every script.
- Limit third-party scripts. Each one costs speed, privacy and reliability.
- **Heavy embeds are the #1 threat to a 98+ score.** Google Maps, YouTube, Calendly, Jobber, HoneyBook and chat widgets all fall in this group.
  - Never place a heavy embed above the fold if it can be avoided.
  - Use a **facade**: show a lightweight image or button and load the real embed only when the visitor clicks it or scrolls near it.
  - For a hero form, build the form natively in HTML and send it to the service (Section 13) instead of embedding a vendor iframe.
- **Popups and modals** open only after the visitor interacts (a scroll, a tap, or exit intent), never on a timer at page load. A timed popup becomes the page's "largest content" and tanks LCP.
- Load Google Analytics (GA4) **after** first paint or on first interaction, not as a render-blocking script at the top of `<head>`.
- Use SVG icons, optimized with SVGOMG. Never use Font Awesome.

9. CSS Performance

- Keep the number of render-blocking stylesheets small. Ideally a page loads `global.css` plus one page file.
- **Minify CSS and JS for production.** Netlify can do this during deploy. Otherwise run a minifier before launch and keep the readable source.
- **Critical CSS** is optional. If PageSpeed flags render-blocking CSS and testing proves a gain:
  - Put only above-the-fold rules in `critical.css`.
  - Defer the rest with the approved pattern and a `<noscript>` fallback:

```
<link rel="stylesheet" href="css/critical.css">
<link rel="stylesheet" href="css/styles.css" media="print"
onload="this.media='all';this.onload=null;">
<noscript><link rel="stylesheet" href="css/styles.css"></noscript>
```

- Animate only `transform` and `opacity` or `filter`. Avoid animating `width`, `height`, `top` or `left`.
- Don't add performance techniques by habit. Measure first, then keep what helps.

10. Performance Targets

Hard rule: every page must score at least the following in **PageSpeed Insights on mobile**:

Category	Target
Performance	**98+**
Accessibility	**100**
Best Practices	**100**
SEO	**100**

Core Web Vital	Target
Largest Contentful Paint (LCP)	under 1.8 s (Google's "good" line is 2.5 s)
Cumulative Layout Shift (CLS)	under 0.05
Total Blocking Time (TBT)	under 100 ms
Interaction to Next Paint (INP)	under 200 ms

- Test **every page**, not just the homepage. Run PageSpeed Insights 2–3 times and use the typical score, because results vary run to run.
- Test the **live deployed URL**. Local scores are not reliable.
- Record before and after scores for each project in `documentation/`.
- The most common reasons a page lands under 98:
  - An oversized hero image.
  - A lazy-loaded hero.
  - A heavy third-party embed above the fold.
  - A timed popup.
  - Too many render-blocking CSS files.
  - Unsubsetted fonts.

Reference: CodeStitch Page Speed Handbook, https://codestitch.app/page-speed-handbook/. Where it differs from this handbook, MigsFlow rules take priority.

11. Accessibility (WCAG 2.2 Level AA)

Why it matters

- In the US, ADA lawsuits over inaccessible websites are filed by the thousands each year, and small businesses are frequent targets.
- The DOJ's 2024 rule requires WCAG 2.1 AA for state and local government sites.
- The European Accessibility Act (in force June 2025) and Ontario's AODA set similar requirements.
- **WCAG 2.2 AA** is the standard to build to. It covers 2.1 and adds more.
- **Don't rely on accessibility overlay widgets.** They don't fix the underlying code, and sites using them still get sued. Accessibility is built into the HTML and CSS.

The four principles (POUR)

Perceivable

- Text contrast is at least **4.5:1**. Large text (24px, or 18.66px bold) and UI parts such as icons, input borders and focus rings need at least **3:1**.
  - Check text over gradients and photos at their *lightest* point.
  - Never put white text on a light accent color.
- Meaningful images have alt text, and decorative images get `alt=""` plus `aria-hidden="true"`. See Section 6.
- Videos have captions. Auto-playing motion longer than 5 seconds has a pause button.
- Never use color as the only signal. An error message also needs an icon or text.
- Text can be resized to 200% and the layout reflows at 320px without horizontal scrolling.

Operable

- Everything works with a keyboard alone, in a logical tab order, and never traps focus. Menus and modals close with Escape.
- Every focusable element shows a **visible focus style** (`:focus-visible`), and the sticky header never hides it. That second part is new in WCAG 2.2 (2.4.11, Focus Not Obscured).
- Include a **skip link** ("Skip to main content") as the first focusable element.
- Tap targets are at least **24×24px**, and 44×44px is better for primary buttons. This is new in 2.2 (2.5.8, Target Size).
- Any action that requires dragging also has a click alternative. This is new in 2.2 (2.5.7, Dragging Movements).
- A hidden mobile menu uses `visibility: hidden`, not just `opacity` or height, so its links can't be tabbed to while it's closed.
- Respect `prefers-reduced-motion`: turn off parallax, autoplay and large movement.
- **Scroll-reveal animations must never hide content with `opacity: 0`.**
  - Scanners and Lighthouse read hidden text as failing contrast.
  - Fade with `filter: opacity()` instead, or keep text visible and animate only `transform`.

- The `h1` never waits on a reveal animation.

Understandable

- Set `<html lang="en">` (or the correct language) on every page.
- Navigation and help links (phone, contact) stay in the same place on every page. This is new in 2.2 (3.2.6, Consistent Help).
- Every form field has a visible `<label>`. A placeholder is not a label.
- Errors are written in text, tied to the field with `aria-describedby`, marked with `aria-invalid="true"`, and announced through an `aria-live` region.
- Don't make people re-enter information they already gave in the same flow. This is new in 2.2 (3.3.7, Redundant Entry).
- Use `autocomplete` attributes on name, email, phone and address fields.

Robust

- Use semantic HTML: `header`, `nav`, `main`, `section`, `article`, `footer`, `button` and `a`. Never use a `div` as a button.
- Use one `h1` per page and never skip heading levels.
- Use ARIA only when HTML can't do the job. Give icon-only buttons an `aria-label`.
- Validate the HTML. Avoid duplicate IDs.

Testing (every page, every launch)

- Lighthouse Accessibility at 100.
- **axe DevTools** or **WAVE** with zero errors.
- Keyboard-only walkthrough: Tab, Shift+Tab, Enter, Space and Escape.
- Test both themes if the site has a dark/light toggle.
- Zoom to 200% and check at a 320px width.
- Do a quick screen reader pass with VoiceOver (Cmd+F5 on Mac): headings, landmarks, forms, menu.

12. SEO and Launch Basics

Every page `<head>`

```
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Move-Out Cleaning San Diego | Barry's Pro Cleaning</title>
<meta name="description" content="Flat-rate move-out cleaning across San Diego County. Deposit-ready kitchens, baths and floors. Get your free quote today.">
<link rel="canonical" href="https://domain.com/services/move-out-cleaning/">
<meta property="og:type" content="website">
<meta property="og:title" content="Move-Out Cleaning San Diego | Barry's Pro Cleaning">
<meta property="og:description" content="Flat-rate move-out cleaning across San Diego County.">
<meta property="og:url" content="https://domain.com/services/move-out-cleaning/">
<meta property="og:image" content="https://domain.com/images/brand/og-image.jpg">
<meta name="twitter:card" content="summary_large_image">
<link rel="icon" href="../../favicon.ico" sizes="32x32">
<link rel="icon" href="../../favicon.svg" type="image/svg+xml">
<link rel="apple-touch-icon" href="../../apple-touch-icon.png">
<link rel="manifest" href="../../site.webmanifest">
<meta name="theme-color" content="#17131a">
```

On-page rules

- **Title:** unique to the page, built as *main keyword + location | Brand*. The brand comes last, except on the homepage.
  - **Aim for 50 characters or fewer. 60 is the hard limit.** Google cuts titles off at about 60 characters, and preview/meta-tag checkers flag anything longer.
  - Count the title as it displays: `&` counts as one character (`&`), and spaces and the `|` separator count too.
  - If it runs long, shorten the keyword part first (drop "in", "and", filler words) and keep the brand name whole.
  - The `og:title` (link previews) follows the same limit and can differ from the page title.
- **Meta description:** unique, about 140–160 characters, with a benefit and a call to action. It doesn't affect ranking directly, but it wins clicks.
- **One** `h1` per page containing the main keyword. Below it, use `h2` and `h3` in order.
- **One main topic per page.**
  - Each service gets its own page targeting "*service + city*."
  - Never make two pages compete for the same keyword.
- **Write for people first.** Use keywords naturally in the title, `h1` , first paragraph, a subheading, image alt text and the URL. Never keyword-stuff.

- **Content that helps:**
  - Real details: prices or starting prices, what's included, process, service area, FAQs and real photos.
  - Thin pages with two sentences don't rank.
- **Internal links:**
  - Every page links to related pages with descriptive anchor text ("see our move-out cleaning checklist," not "click here").
  - No orphan pages.
- **No broken links.** Check before launch with a link checker (e.g. the W3C Link Checker or the "Broken Link Checker" browser extension).
- **Image SEO:** use descriptive file names, alt text, and compressed WebP.
- **Mobile-friendly and fast:** Google ranks the mobile version, so Sections 5 and 10 are SEO rules too.

Local business essentials

- **NAP consistency:** the business **N**ame, **A**ddress (or service area) and **P**hone appear the same way in the footer, on the Contact page, in the schema and on the Google Business Profile.
- **Phone numbers are tap-to-call links:** `<a href="tel:+16195550100">`.
- **Google Business Profile:** link it to the new site, and add a link to the Google reviews.
- **LocalBusiness schema (JSON-LD)** on the homepage. Use the most specific type available, such as `HouseCleaningService`, `LegalService`, `MovingCompany` or `ProfessionalService`.

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "HouseCleaningService",
  "name": "Business Name",
  "url": "https://domain.com/",
  "logo": "https://domain.com/images/brand/logo.png",
  "image": "https://domain.com/images/brand/og-image.jpg",
  "telephone": "+1-619-555-0100",
  "email": "hello@domain.com",
  "address": { "@type": "PostalAddress", "addressLocality": "San Diego", "addressRegion": "CA",
"addressCountry": "US" },
  "areaServed": "San Diego County",
  "openingHours": "Mo-Su 08:00-21:00",
  "sameAs": ["https://facebook.com/...", "https://instagram.com/..."]
}
</script>
```

- **Optional schema:**
  - `Service` on service pages.
  - `BreadcrumbList` on nested pages.

    - `FAQPage` where there's an FAQ.
- Only mark up content that's actually visible on the page, and never add fake ratings.
- Validate with Google's **Rich Results Test**.

Site files

- **Favicon set at the root:**

File	Size
`favicon.ico`	32×32
`favicon.svg`	—
`apple-touch-icon.png`	180×180
`icon-192.png`	192×192
`icon-512.png`	512×512

  Also add a `site.webmanifest` that lists the two PNG icons, the name and the theme color. RealFaviconGenerator.net produces the full set.

- **Open Graph image:** `images/brand/og-image.jpg` at 1200×630, under 300 KB, showing the brand. It appears when the link is shared by text or on social media.
- `robots.txt`:

```
User-agent: *
Allow: /
Sitemap: https://domain.com/sitemap.xml
```

- `sitemap.xml`: list every public page's full URL with a trailing slash. Update it whenever a page is added.
- `404.html`: a branded "page not found" page with the nav and a link home. Netlify and GitHub Pages use it automatically.

Redirects

- When replacing an old site, or moving away from `html/` URLs, map every old URL to its new one so rankings and bookmarks carry over.
- On **Netlify**, put the redirects in a `_redirects` file at the root:

```
/html/about.html    /about/      301
/html/contact.html  /contact/    301
/about-us           /about/      301
```

- GitHub Pages can't do server redirects. If a GitHub-hosted site needs them, move it to Netlify before launch.

After launch

- Verify the site in **Google Search Console**, submit the sitemap and request indexing for the main pages.
- Install GA4 (Section 8), or keep the client's existing GA4 ID so their history carries over.
- Update the website link on the Google Business Profile, Facebook, Instagram, Yelp and any directories.

13. Forms and Third-Party Integrations

MigsFlow builds the front end. Forms get sent to an outside service, so there is no custom server.

Choosing where a form sends

Need	Use
Simple contact form that emails the client	**Netlify Forms** (on Netlify sites), **Web3Forms** or **Formspree**
Leads must land in the client's CRM (Jobber, HoneyBook, HubSpot)	The CRM's own form embed, **or** a native form that posts to a **Zapier/Make webhook**, which then creates the lead in the CRM
Booking	The client's booking tool (Calendly, Jobber, Acuity), behind a facade (Section 8)

Rules

- **Never put API keys or secrets in front-end code.**
  - Anything in HTML or JS is public.
  - Public form keys that the service designed to be public (Web3Forms access keys, Formspree IDs) are fine.
  - Private CRM API keys are never fine. Use Zapier or Make in the middle instead.
- **Contact form and quote form are separate** when a client needs both. The quote or lead form goes to the CRM, and the contact form goes to email.
- Every form has:
  - Visible labels, the `required` attribute and the right `type` and `autocomplete` values.
  - Accessible error messages (Section 11).
  - A **honeypot** field for spam: a hidden field that bots fill in and people never see. Add the service's spam setting too.
  - A clear thank-you state or page, plus GA4 conversion tracking on success.
  - **A real test submission before launch**, checked all the way to the inbox or CRM.
- **Multi-step and qualifying forms:**
  - The branching logic (which questions to show, and who qualifies) lives in the page's JavaScript.
  - Pass the chosen service through the URL (`quote/?service=move-out`) so a "Book now" button preselects it.
  - The form still works, and still submits, if JavaScript fails. At minimum it submits all fields.
- **Phone numbers and texting:**
  - If the client will text leads, add SMS consent wording beside a checkbox, for example: "By checking this box, you agree to receive text messages from *Business*. Msg & data rates may apply. Reply STOP to opt out."
  - The checkbox is never pre-checked.

- **Privacy Policy page** is required whenever a form collects personal info, when analytics run and when SMS consent is collected. Link it in the footer.
- **Document every integration** in `documentation/` (vendor, account owner, where submissions go, any monthly cost), so the client and the next developer know how it works.

14. Domains, Hosting, Security and Git

Domains and DNS

- When pointing a domain at the new site, **change only the website records** (the A record or CNAME for `@` and `www`).
- **Never touch MX, TXT/SPF, DKIM or DMARC records.** They run the client's email.
- Screenshot the existing DNS records before changing anything.
- Make sure both `domain.com` and `www.domain.com` work. One redirects to the other. HTTPS is on (Netlify and GitHub Pages issue free certificates).

Security basics

- HTTPS only. No "mixed content" (`http://` images or scripts on an `https://` page).
- External links that open in a new tab use `target="_blank" rel="noopener"`.
- Use embeds and scripts only from trusted sources, with pinned versions.

Git

- One repo per project, with `main` as the default branch.
- Write short, clear commit messages describing what changed.
- Include a `.gitignore` in every project:

```
.DS_Store
node_modules/
documentation/private/
*.log
```

- **Never commit private client material:**
  - Contracts, logins, invoices and personal info belong in `documentation/private/` (ignored) or outside the repo.
  - Everything in a public repo, and everything deployed, is public.

15. Content Integrity

- Reviews and testimonials are **real**, quoted accurately and credited the way the source shows them. Never write fake reviews.

- Ratings, review counts, years in business, "clients served" and awards come from the client or a verifiable source.
- Use only licensed, client-provided or properly credited images and fonts.
- Mark placeholder content clearly (e.g. `[PLACEHOLDER: team photo]`) and search for it before launch so nothing ships by accident.
- The copyright year in the footer updates automatically through `global.js`.

16. MigsFlow Build Workflow

1. **Structure:** root `index.html`, page folders, `css/`, `javascript/`, `images/`, `fonts/`, `documentation/`, `.gitignore`, `CLAUDE.md`.
2. **Global system:** fonts, tokens, reset, typography, buttons, nav, footer, and accessibility utilities in `global.css` and `global.js`.
3. **Semantic HTML:** build each page with landmarks, one `h1` and clean URLs. Keep CSS and JS out of the HTML.
4. **Page styling:** only unique styles in page files, written mobile-first.
5. **Behavior:** shared scripts in `global.js` and page behavior in page files, all deferred.
6. **Assets:** crop, compress and convert to WebP with `-m` versions. Subset and self-host fonts. Optimize SVGs.
7. **Loading strategy:** preload the hero and the main font, lazy-load below the fold, decode asynchronously, and put facades on heavy embeds.
8. **SEO:** titles, descriptions, canonicals, Open Graph, schema, favicon set, `robots.txt`, `sitemap.xml`, `404.html` and redirects.
9. **Forms:** connect to the service, add a honeypot, run a test submission and add a Privacy Policy.
10. **Test:** responsive widths, keyboard, axe/WAVE, contrast, both themes, console errors and broken links.
11. **Clean and minify:** remove dead code, duplicates, `console.log` calls and placeholders. Minify for production.
12. **Final performance pass:** PageSpeed Insights mobile on **every page** must reach the Section 10 targets. Record the scores.

17. Launch Checklist (Definition of Done)

Code

- [ ] No inline CSS or JS beyond the approved exceptions (Section 3).
- [ ] Shared code lives in global files with no duplicate styles or functions.
- [ ] Mobile-first CSS, with no horizontal scroll from 320px to 1920px.
- [ ] Clean folder URLs with trailing slashes, and all relative links working.
- [ ] No console errors, no `console.log`, no placeholder text.
- [ ] Back-to-top button on every page, bottom-right, appearing on scroll.

Assets

- [ ] WebP with `-m` crops, explicit dimensions, `decoding="async"`, lazy-loading only below the fold, and a preloaded hero.
- [ ] Self-hosted, subsetted fonts with `font-display: swap`. Only the needed weights are loaded.
- [ ] SVG icons are optimized. No Font Awesome and no jQuery.

Performance (mobile, live URL, every page)

- [ ] Performance 98+, Accessibility 100, Best Practices 100, SEO 100.
- [ ] No timed popups and no heavy embeds above the fold.

Accessibility

- [ ] WCAG 2.2 AA: contrast, keyboard, focus visible, skip link, labels, 24px targets, reduced motion.
- [ ] axe/WAVE reports zero errors, and the VoiceOver pass is done.

SEO

- [ ] Unique title (50 characters or fewer, 60 max), description, canonical, Open Graph and one `h1` on every page.
- [ ] LocalBusiness schema validated. Favicon set, OG image, `robots.txt`, `sitemap.xml` and `404.html` in place.
- [ ] Redirects added for old URLs, and no broken links.

Forms and integrations

- [ ] Every form tested end to end, honeypot added and thank-you state working.
- [ ] No secrets in the code. Integrations are documented.
- [ ] Privacy Policy linked, and SMS consent included if the client texts leads.

Launch and handoff

- [ ] Domain points to the new site with HTTPS on both `www` and the bare domain. Email records are untouched.
- [ ] Search Console verified and sitemap submitted. GA4 connected and the client has access.
- [ ] Google Business Profile and social links updated.
- [ ] Before/after PageSpeed scores and integration notes saved in `documentation/`.
- [ ] The client has a short care guide: how to request changes, plus where the domain, hosting and forms live.

18. Instructions for AI Coding Agents

When working on a MigsFlow website, follow this handbook as the default standard.

Before writing code:

- Read the existing structure and reuse the global styles, tokens and functions that are already there.
- Decide whether new code is global or page-specific.
- Build with HTML, CSS and vanilla JS before reaching for any dependency.
- Keep the established naming, paths and responsive behavior.

Never silently:

- Change the architecture.
- Add frameworks, font CDNs or icon libraries.

- Use an `html/` folder or `.html` page URLs.
- Put CSS or JS inside HTML beyond Section 3.
- Duplicate reusable code.
- Invent reviews or facts.
- Put secrets in the front end.

If a request conflicts with a hard rule, explain the conflict and the trade-off before building an exception.

Keep changes focused. Don't rewrite unrelated code unless asked. When reporting changes, say which file each change is in.

Before calling work done, check it against the Section 17 checklist and report anything not yet verified.